



PASCAL

Programming Language

Contents

I.	La cámara de gravedad — Forja saiyajin	3
I.	Prólogo	5
II.	Introducción	6
A.	Ejercicio 0: El Despertar del Príncipe.....	7
B.	Ejercicio 1: Calibrador de Gravedad	8
C.	Ejercicio 2: Detección de Ki con Scouter	9
D.	Ejercicio 3: Raciones de Semillas Senzu	10
E.	Ejercicio 4: Validación de Scouter (Lógica Booleana)	11
F.	Ejercicio 5: Sobrecarga de Ki (Integer Overflow)	12
G.	Ejercicio 06: La Trampa de Babidi	13
H.	Ejercicio 7: Rango de Detección del Scouter (Constantes y Precedencia) ..	14

I. La cámara de gravedad — Forja saiyajin

Instrucciones Generales

- ✓ **Versión:** Free Pascal Compiler (FPC) 3.2.x o superior (se recomienda 3.2.2). El código debe compilar de forma nativa desde terminal mediante el comando *fpc*.
- ✓ **Estructura:** un directorio independiente por cada ejercicio (ex00/, ex01/, etc.); cada programa debe ser autosuficiente, con su propio bloque program, o ser una unit si así lo exige el enunciado.
- ✓ **Linter Obligatorio:** indentación estricta de 2 o 4 espacios; uso riguroso de punto y coma (;) para delimitar sentencias; omitir estrictamente el punto y coma antes de else; delimitar explícitamente los bloques lógicos mediante begin/end; y garantizar que todos los archivos compilen sin advertencias (warnings) ni errores.
- ✓ **Comentarios:** obligatorios y redactados exclusivamente en inglés.
- ✓ **Nomenclatura**
 - **Archivos:** snake_case, extensión en minúsculas.
 - **Tipos definidos por el usuario:** prefijo t + PascalCas.
 - **Punteros a tipos:** prefijo p (o ^t) (ej. pNodo = ^tNodo).
 - **Constantes:** mayúsculas con guion bajo.
 - **Variables y subprogramas:** snake_case consistente.
- ✓ **IDE** – La versión que queráis, tanto como visual, eclipse o Lazarus.

✓ **Compilación**

Verificación del compilador: *fpc -iV* (debe devolver 3.2.x o superior).

Compilación estándar: *fpc -Co -Cr -Mobjfpc nombre_archivo.pas*

- **-Co**: comprobación de desbordamiento de enteros.
- **-Cr**: comprobación de límites de arrays y rangos.
- **-Mobjfpc**: sintaxis moderna estándar de **Free Pascal**.

Directivas de seguridad activas en tiempo de ejecución:

- **{SR+}** — comprobación de límites de rangos y arrays.
- **{Q+}** — detección de desbordamiento aritmético.

Depuración de memoria (fugas con punteros): *fpc -gh nombre_archivo.pas*

(-gh vincula la unidad heaptrc, que reporta en consola cualquier bloque de memoria no liberado al finalizar la ejecución).

Configuración

- Verificar la versión del compilador con **fpc -iV**.
- Compilar cada ejercicio con el comando estándar (**-Co -Cr -Mobjfpc**).
- Activar **{SR+}** y **{Q+}** en el código fuente para validaciones en tiempo de ejecución.
- En ejercicios con punteros, compilar además con **-gh** para verificar ausencia de fugas de memoria mediante **heaptrc**.

Entrega y Evaluación

- Entrega en repositorio Git.

I. Prólogo

¡Escúchame bien, insecto! Has osado cruzar las puertas de la Cámara del Tiempo y la Gravedad Aumentada. Crees que por haber convalidado materias ya eres un guerrero de élite, pero en este terreno eres un guerrero de clase baja que no duraría ni un segundo frente a Freezer. Aquí no valen trucos baratos ni lenguajes débiles de tipado blando. Esto es Pascal: el código de la realeza Saiyajin, inflexible, riguroso y sin compasión.

Si un solo punto y coma falta, la gravedad de la nave de entrenamiento se descalibrará y quedarás aplastado contra el suelo. Si no controlas el desbordamiento de memoria, tu propio ki destruirá tu cuerpo antes de alcanzar el Super Saiyajin. No tolero excusas, advertencias del compilador ni debilidades. Si tu código no compila al primer intento, tu calificación será un miserable cero. Demuéstrame que llevas la sangre de un verdadero príncipe Saiyajin en tus venas o lárgate de mi nave.



II. Introducción

Para sobrevivir a esta sesión, tus manos deben obedecer a la consola antes de que tu mente dude.

Cada instrucción que escribas dentro de la nave será evaluada por el núcleo de control bajo tres mandamientos inquebrantables: tipado estricto sin conversiones automáticas, delimitación limpia con bloques **begin** y **end**, y cálculo exacto de cada celda de memoria para evitar que tu energía colapse el **scouter**. Activa tu terminal, carga el compilador y no des un solo paso en falso.

En este primer nivel forjarás las bases de hierro que exige la arquitectura de Pascal:

- **Estructura elemental:** cabecera program, declaración obligatoria de variables en el bloque var, y cuerpo delimitado por begin y end.
- **Tipos primitivos:** enteros con signo (ShortInt, SmallInt, Integer, LongInt), coma flotante (Real), caracteres individuales (Char) y booleanos (Boolean).
- **Aritmética entera:** operadores div (cociente entero) y mod (resto o módulo).
- **Control de memoria y directivas del compilador:** activación de comprobaciones en tiempo de ejecución mediante {\$Q+} (detección de *overflow* aritmético) y {\$R+} (comprobación de rangos).



A. Ejercicio 0: El Despertar del Príncipe

- **Directorio:** ex0/
- **Archivo:** saiyan_power.pas
- **Funciones autorizadas:** writeln

Objetivo: Escribir tu primer programa en Pascal. El príncipe Vegeta ha activado su scouter y necesita mostrar el mensaje que anuncia el inicio de su entrenamiento. Tu misión es imprimir exactamente el texto solicitado en la terminal.

Salida Esperada

El programa debe mostrar el siguiente mensaje:

```
I am the Prince of all Saiyans! Training starts now.
```

No debes solicitar datos al usuario ni realizar cálculos. Solo necesitas utilizar la instrucción `writeln`. El programa debe compilar correctamente como un archivo `.pas`.

B. Ejercicio 1: Calibrador de Gravedad

- **Directorio:** ex1/
- **Archivo:** gravity_level.pas
- **Funciones autorizadas:** readln, writeln

Objetivo: La cámara de gravedad comienza en un nivel introducido por el usuario, expresado en **G** como un número entero.

El nivel máximo de entrenamiento es de **400G**.

El programa debe:

1. Leer el nivel actual de gravedad.
2. Calcular cuántos G faltan para alcanzar los 400G.
3. Mostrar el nivel actual.
4. Mostrar cuántos G faltan para alcanzar el nivel máximo.

Salida Esperada

```
Current gravity (G): 150
System: You need 250G more to reach maximum training level!

Current gravity (G): 100
System: You need 300G more to reach maximum training level!

Current gravity (G): 400
System: You need 0G more to reach maximum training level!
```

C. Ejercicio 2: Detección de Ki con Scouter

- **Directorio:** ex2/
- **Archivo:** scouter_calc.pas
- **Funciones autorizadas:** readln, writeln

Objetivo: Leer el nivel de ki base de un adversario (tipo Real). El scouter calcula el multiplicador del Ōzaru (mono gigante), que es exactamente 10.0 veces el ki base. La salida debe mostrarse estrictamente formateada con 2 decimales (:0:2).

Misión:

- Declara una variable de tipo Real para el ki base.
- Lee el valor introducido por el usuario con ReadLn.
- Calcula el ki del Ōzaru multiplicando ese valor por 10.0.
- Imprime el resultado usando el formato: 0:2 para forzar exactamente 2 decimales.

Salida Esperada

```
Enter base Ki: 1200.45
Ozaru Ki multiplier: 12004.50
```

D. Ejercicio 3: Raciones de Semillas Senzu

- **Directorio:** ex3/
- **Archivo:** senzu_distributor.pas
- **Funciones autorizadas:** readln, writeln

Objetivo: El maestro Karin entrega un saco con N semillas senzu (Integer). Cada guerrero del escuadrón recibe exactamente 6 semillas. Utilizando obligatoriamente los operadores div y mod, calcula cuántos guerreros reciben su ración completa y cuántas semillas sobran en la reserva.

Misión

- Declara una variable Integer para el total de semillas (N).
- Lee el valor introducido por el usuario con ReadLn.
- Usa div para calcular cuántos guerreros reciben ración completa (grupos de 6).
- Usa mod para calcular cuántas semillas sobran (el resto de esa división).
- Imprime ambos resultados por consola.

Salida Esperada

```
Total Senzu Beans: 50
Warriors supplied: 8
Beans remaining in bag: 2
```

E. Ejercicio 4: Validación de Scouter (Lógica Booleana)

- **Directorio:** ex4/
- **Archivo:** threat_check.pas
- **Funciones autorizadas:** readln, writeln

Objetivo: Leer el nivel de ki de un enemigo (Integer). Si el ki es estrictamente superior a 9000, una variable de tipo Boolean debe tomar el valor TRUE, de lo contrario FALSE. Imprimir el valor booleano resultante directamente por consola.

Misión:

- Declara una variable Integer para el ki del enemigo.
- Lee el valor introducido por el usuario con ReadLn.
- Declara una variable Boolean y asígnale directamente el resultado de la comparación ($ki > 9000$).
- Imprime esa variable booleana con WriteLn.

Salida Esperada

```
Enemy Ki: 9001
It is over 9000: TRUE

Enemy Ki: 4000
It is over 9000: FALSE
```

F. Ejercicio 5: Sobrecarga de Ki (Integer Overflow)

- **Directorio:** ex5/
- **Archivo:** ki_overflow.pas
- **Funciones autorizadas:** writeln

Objetivo: Demostrar cómo colapsa la memoria estática por desbordamiento sin directivas de protección. Declara una variable de tipo ShortInt (rango -128 a 127). Asigna el valor 127, súmale 1 e imprime el resultado obtenido.

Misión

- Declara una variable de tipo **ShortInt**.
- Asígnale el valor límite superior del tipo (127).
- Súmale 1 a esa misma variable.
- Imprime el resultado con **WriteLn**.

Nota: este ejercicio no lee nada del usuario (**readln** no está entre las funciones autorizadas) — todos los valores están fijos en el código.

Salida Esperada

```
Current Ki capacity: 127
Charging 1 unit of Ki...
Scouter exploded! Current value: -128
```

Problema Curioso: El Escudo de los Dioses (Directivas **{\$Q+}** y **{\$R+}**)

*Este es el gancho hacia el ejercicio 06: una vez veas que ki_overflow.pas imprime un valor incorrecto sin avisar de nada, la pregunta que debes hacerte es ¿cómo le digo al compilador que no permita esto silenciosamente? — la respuesta son esas dos directivas que ya vimos en la teoría (**{\$Q+}** para overflow aritmético, **{\$R+}** para rango), que es justo lo que trabajarás en el ejercicio 06.*

G. Ejercicio 06: La Trampa de Babidi

- **Directorio:** ex06/
- **Archivo:** divine_shield.pas
- **Funciones autorizadas:** writeln

Objetivo: Toma el mismo código del ex05, pero añade en la primera línea del archivo las directivas de compilación **`{ $Q+ }` (*Overflow Checking*)** y **`{ $R+ }` (*Range Checking*)**. Compila y ejecuta el programa.

El misterio técnico: En el ex05, Pascal permite silenciosamente que el valor salte de 127 a -128 debido a la aritmética modular en complemento a dos. En sistemas críticos (un simulador de vuelo, una base militar), ese tipo de fallo silencioso provoca catástrofes reales.

Misión

- Copia el código de ki_overflow.pas (ex05).
- Añade `{ $Q+, R+ }` como primera línea del archivo, antes de `program ....`
- El resto del código queda exactamente igual: `ShortInt`, asignación de 127, suma de 1, `WriteLn`.

Salida Esperada: El programa ya no imprime -128. El compilador inyecta código de salvaguarda y, al ejecutar, el sistema aborta con un error en tiempo de ejecución.

```
Current Ki capacity: 127
Charging 1 unit of Ki...
Runtime error 201 at $0040162E
$0040162E
$00407797
```

Concepto clave: con las directivas activas, el programa **sigue compilando sin problema** — el error no aparece hasta que se *ejecuta* la línea problemática (`x := x + 1`). Es un error de **runtime**, no de compilación. Esa es la diferencia que debes poder explicar si te preguntan en la uni.

H. Ejercicio 7: Rango de Detección del Scouter (Constantes y Precedencia)

- **Directorio:** ex7/
- **Archivo:** scouter_range.pas
- **Funciones autorizadas:** readln, writeln

Concepto clave: Declaración obligatoria de constantes en la sección const y uso estricto de paréntesis en expresiones booleanas compuestas.

Objetivo: Declarar dos constantes: MIN_KI = 500 y MAX_KI = 8500. El programa solicita el ki de un guerrero (Integer) y debe evaluar en una variable booleana si el valor se encuentra **dentro del rango medible** (ambos límites inclusive) usando and.

Misión

- Declara MIN_KI y MAX_KI en la sección const, antes del var.
- Declara una variable Integer para el ki del guerrero.
- Lee el valor con ReadLn.
- Declara una variable Boolean y asígnele el resultado de comprobar que el ki está entre MIN_KI y MAX_KI (ambos inclusive), combinando dos comparaciones con and.

Salida Esperada

```
Enter target Ki: 1200
Target in scouter range: TRUE

Enter target Ki: 350
Target in scouter range: FALSE
```